

Intro til Git

En introduksjon til git og hvordan det kan brukes



Hvorfor git?

For meg har git to hovedformål

- 1 Holde en (lesbar) historie over hva som har blitt gjort
- 2 Gjøre det mulig for flere personer å samarbeide om kode

Conventional commits

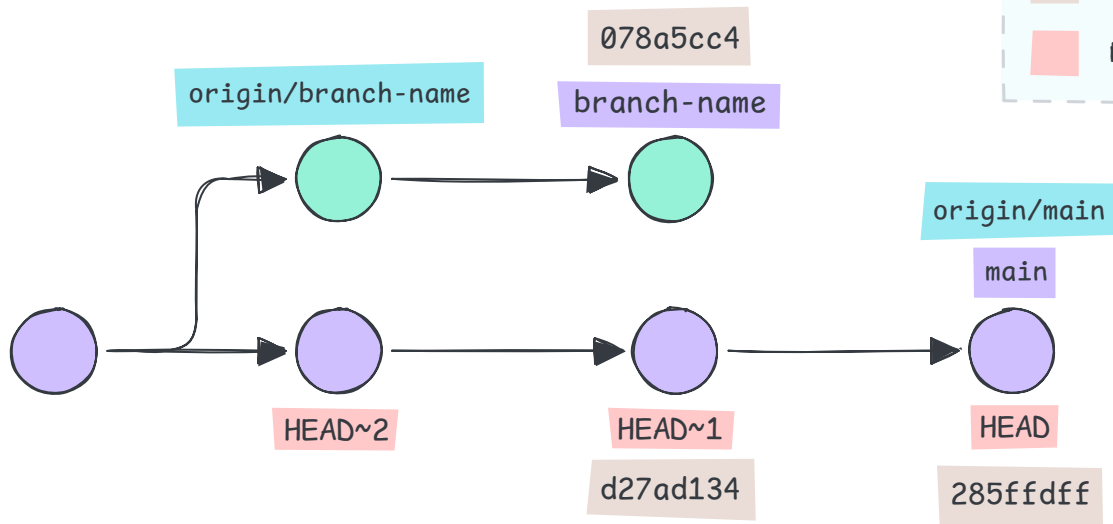
feat(service): Add pre-processing stage

We added a pre-processing stage to the system so that we can handle new features introduced by upgrading the ground segment.

This has the potential side effect of delaying data storage, but it makes up for that by decreasing the total data stored which should speed up retrieval.

Refs: closes #16

Referanser i git-systemet

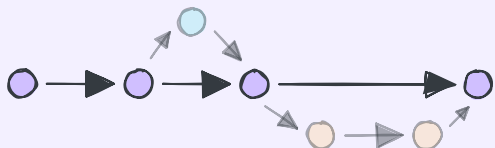


- Lokale branches
- Remote branches
- Absolutte referanser
- Relative referanser

Forskjellige filosofier – rebase vs merge

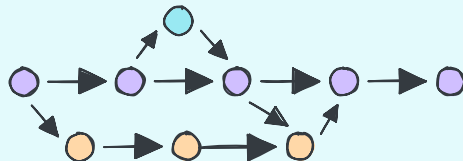
Rebase arbeidsflyt

- Endringer er kronologisk til når de ble ferdige
- Ingen merge commits
- Lineær historikk



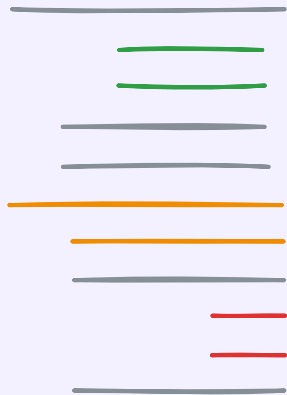
Merge arbeidsflyt

- Endringer har tidspunktene for når de ble gjort
- Merge commits
- Branching historikk



Filenes tilstand – disk, staging, commit

Disk



Filen slik den er på
harddisken

Staging



Filen slik den ser ut i
staging, venter på commit

Commit

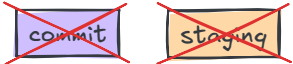


Filen slik den er lagret i
git historikken

Vanlig bruk – staging og commits

Kommando	Beskrivelse	Resultat
diff, status	Informasjon om filer og hva status er i stadiene	
add	Legg til en fil i staging	
commit	Lag en commit med det som er i staging	

Vanlig bruk – staging og commits

Kommando	Beskrivelse	Resultat
<code>restore --staged</code>	Angre ting i staging, men ta vare på endringen lokalt	
<code>restore</code>	Slett endringen helt	
<code>reset</code>	Slett en commit, bevar endringene på disk	
<code>reset --hard</code>	Slett en commit, ikke ta vare på endringen, start på nytt	

Vanlig bruk – remotes

En remote er en separat kopi av git repoet, med sine egne branches og tags. Vanligvis har man én av disse, den heter origin og peker til GitLab for oss.

Kommando	Beskrivelse
fetch	Synkronisere tags fra remote slik at e.g. origin/my-branch peker på samme commit som på serveren. Endrer ingenting.
pull	Hent endringer fra remote (helst ff-only).
push	Send endringer du har lokalt til remote

Vanlig bruk – branches

Branches tillater en å jobbe på flere ting samtidig. Det kan også virke som et sted man kan jobbe før det er helt ferdig. Kan lett skape mye rot og mye hierarki om det ikke brukes fornuftig.

Kommando	Beskrivelse
switch	Bytt til en annen branch, lokale endringer kan ikke være i konflikt (se stash).
switch -c	Lag en ny branch
stash	Lagre endringer i en midlertidig commit, hentes ut med stash pop/apply

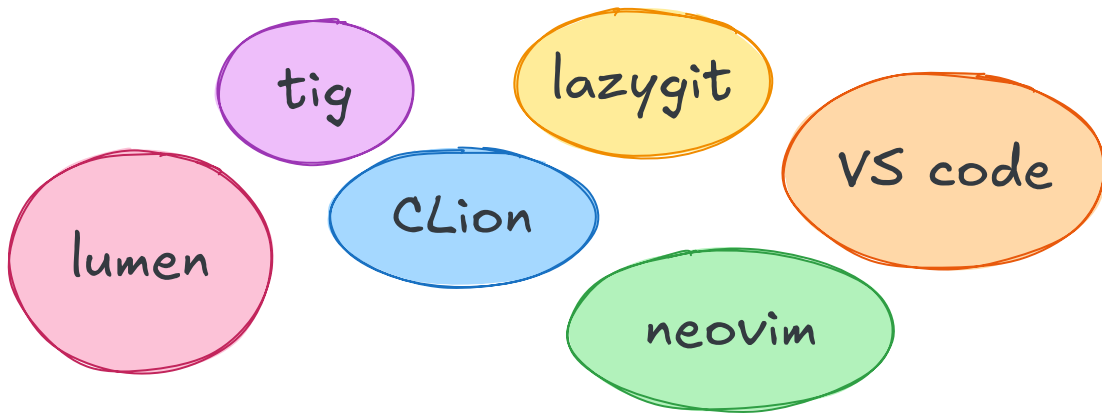
Kommandoer for ren historikk

Jeg foretrekker at alle branches kun inneholder én enhet med arbeid, helst betyr det også at de kun har én commit.

Kommando	Beskrivelse
<code>commit --amend (--no-edit)</code>	Oppdater forrige commit med det du har endret.
<code>rebase origin/main</code>	Flytt branch til å basere seg på origin/main
<code>rebase -i [commit-id]</code>	Gir deg mulighet til å endre historikken
<code>push --force(-with-lease)</code>	Sender endringer til remote selv om de overskriver
<code>commit --fixup [commit-id]</code>	Lager en fixup commit for å slippe å skrive en kommentar

Verktøy – commits og inspeksjon

Jeg syntes verktøy er mest nyttige for å se over det man har gjort, inspisere tidligere endringer, og velge hva som skal med i en commit.



Git config

Git config er der man legger git konfigurasjon. Det finnes global config som gjelder for alle prosjekter, og lokal config som kun gjelder det ene prosjektet.

Global config

Kommando `git config --global`

Fil `${HOME}/.gitconfig`

Lokal config

Kommando `git config`

Fil `${REPO_ROOT}/.git/config`

Git config

```
[user]
  name = Aleksandra Glesaaen Ødegård
  email = aleksandra@glesaaen.com
  signingkey = "$HOME/.ssh/signing_key.pub"
```

```
[push]
```

```
  default = simple
```

```
  autoSetupRemote = true
```

```
[commit]
```

```
  gpgsign = true
```

```
[gpg]
```

```
  format = ssh
```

```
[pull]
```

```
  ff = only
```

➡ Kun push branch du er på

➡ Automatisk lag branch på remote

➡ Signer alle commits

➡ Kun tillat pull hvis den er FF

<https://blog.gitbutler.com/how-git-core-devs-configure-git>

Min arbeidsflyt

- 1 Velg eller lag en issue for oppgaven jeg skal gjøre
- 2 Lag en branch for arbeidet, e.g. feat/spectrum-capture
- 3 Når noe er klart, skriv en god commit-melding for arbeidet
- 4 Hver gang jeg endrer noe før andre har sett på det gjør jeg
`git commit --amend --no-edit`
- 5 Etter andre har sett på det lager jeg --fixup commits
- 6 Før det skal merges gjør jeg
`git rebase --autosquash origin/main`